# Your camera roll is a graveyard of intentions.

We screenshot the concert we want to go to, the book a friend loved, the chart we meant to read up on. Then nothing happens. The screenshot sits there.

Turning a screenshot into an action today means retyping it, or uploading it to someone's cloud.

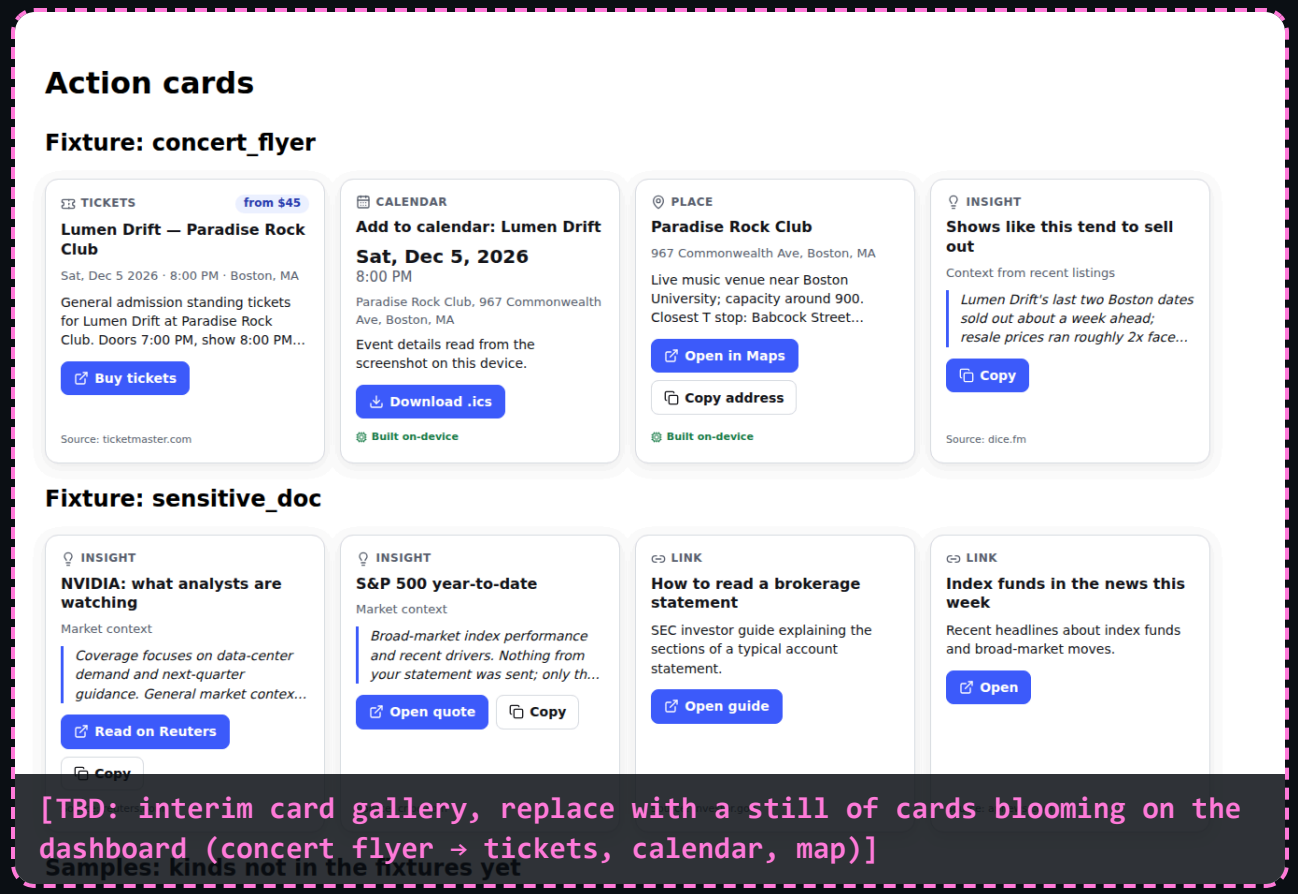
**Cloud assistants don't work on a work laptop**

- The whole image goes to a remote server: names, emails, internal numbers and all.
- NDAs and compliance policies often forbid sending internal material to outside services.
- So the screenshots that matter most at work are the ones you can't send anywhere.



Everyone screenshots things they mean to act on, then never does. Cloud assistants could help, but they upload the whole image, with names, emails and internal numbers. On a work laptop that can break an NDA or a compliance policy, so the screenshots that matter most at work can't go to them.

# Drop a screenshot. Get actions back.
# The image never leaves your laptop.




[TBD: interim card gallery, replace with a still of cards blooming on the dashboard (concert flyer → tickets, calendar, map)]

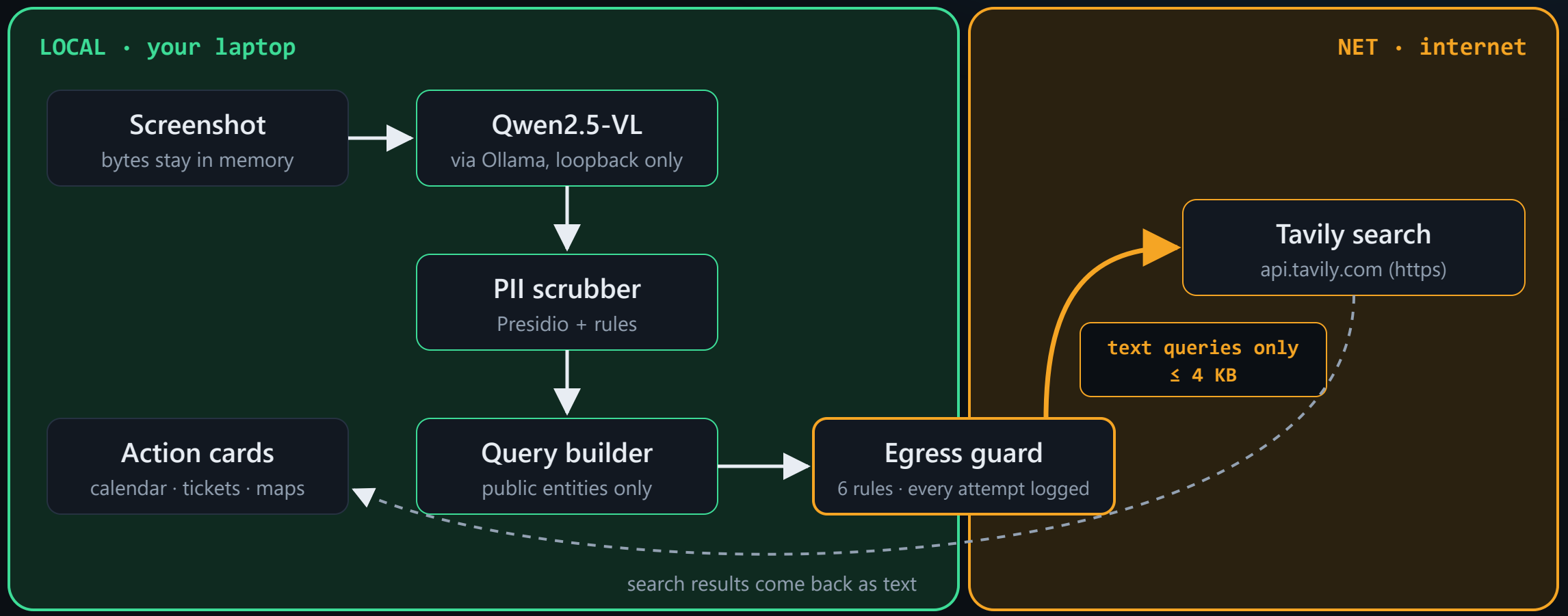
A local vision model reads the screenshot on your machine. Private details are removed. Only a few short public text queries go to the web.

Results come back as cards you can act on: **Add to calendar**, buy tickets, open in Maps.



Latent turns a screenshot into actions without the image leaving your laptop. A local model reads it, private details are stripped, and only short public text queries go to the web. Results bloom as cards you can act on. HAND-OFF: switch to the live demo now (concert flyer, then air-gap). Come back for slide 3.

# Two zones. One door between them.






Green is your laptop: Qwen2.5-VL reads the image through Ollama on loopback, the scrubber strips PII, the query builder keeps only public entities. One arrow crosses: up to three short text queries, each request at most 4 KB. The egress guard runs on your laptop, right at the boundary: it is the only module allowed to open a connection, it logs every attempt, and the request crosses only after it passes.

# No image or raw text leaves the device — enforced by code and shown in the audit log.

### Eval · `eval/results.md`

| | |
|---|---|
| Images | **10** synthetic, fake PII |
| Intent accuracy | [TBD: from eval/results.md] |
| Leaks | [TBD: exact count, do not round] |
| Scrubber off | [TBD: leaks with --no-scrub] |

### Egress guard · first failure blocks

1. Air-gap on (`LATENT_OFFLINE`) blocks everything
2. Allow-listed host, https, default port
3. Body ≤ 4096 bytes (an image cannot fit)
4. No base64-looking run of 200+ chars
5. No PII found in any string in the body
6. No API key in the body (headers only)

Source: `backend/app/egress.py`; a test fails if any other module opens a network client.

[TBD: air-gap demo still internal email: extraction done locally, search ✕ blocked in the audit log]



We don't claim magic, we show it. Ten fake-PII screenshots: intent accuracy [TBD], leaks [TBD exact]. Six egress rules in egress.py, and a test fails if any other module opens a connection. You saw air-gap: extraction finished locally, search blocked, and the audit log lists every outbound request with its byte count.

# Free for people. Paid for companies.

### B2C Affiliate share on the action

- Ticket, shopping and book cards are where intent turns into a purchase.
- Every commercial link already passes through one hook, `affiliate.tag_url`, where a partner tag goes.
- Revenue only on links the user chooses to click; the app stays free.

### B2B Enterprise tier (planned)

- **On-prem model**: the vision model runs on company hardware.
- **Policy controls**: admins set the egress allow-list, scrubber rules and air-gap defaults.
- **Audit-log export**: every outbound request, sent or blocked, for compliance review.



Two lines of revenue. Consumers get it free; when they buy tickets, products or books from a card, an affiliate tag earns a share. The hook, affiliate.tag_url, exists today and is a pass-through. Companies would pay for a planned enterprise tier: on-prem model, admin policy controls and audit-log export for compliance.

# Why Latent fits each track

| TRACK | HOW WE MEET IT |
|---|---|
| **Privacy-First** | Vision runs locally; one egress module with six rules; air-gap switch; audit log of every outbound request and its size; leak eval on fake-PII screenshots. |
| **SURGE · Tavily & Exa** | Tavily search turns scrubbed public entities into ticket, shop, book and map cards. Exa related-content cards are our stretch goal (issue #19). |
| **Most Startup-able** | A problem everyone has, a free consumer app with affiliate revenue, and an enterprise tier for teams that can't use cloud assistants. |

### Team

[TBD: names and roles]

### What's next

Opt-in auto-ingest from the screenshot folder, a local intent library, Exa related-content cards.

### Our ask

[TBD: the ask, e.g. pilot teams or feedback]



Privacy-First: local vision, one guarded door, air-gap, audit log, leak eval. SURGE: Tavily turns scrubbed entities into real actions; Exa is our stretch. Startup: everyone has this problem, consumers are free, enterprises pay. Next: opt-in auto-ingest, a local intent library, Exa cards. Our ask: [TBD]. Thank you.